Better specification of alignment

Document #:	P4192R0 [Latest] [Status]
Date:	2026-04-20
Project:	Programming Language C++
Audience:	Core Working Group

Reply-to:	Vlad Serebrennikov <serebrennikov.vladislav@gmail.com>

Contents

- 1 Abstract
- 2 Revision history
- 3 Implementation divergence
- 4 Issues with status quo wording
- 5 Approach
- 6 Proposed wording
  - 6.1 Alignment [basic.align]
  - 6.2 Alignof [expr.alignof]
  - 6.3 Attribute syntax and semantics [dcl.attr.grammar]
  - 6.4 Alignment specifier [dcl.align]

1 Abstract

There are multiple open Core issues against 6.8.3 [basic.align] (1211, 2840) and 9.13.2 [dcl.align] (1617, 2223, 3024). This proposal aims to improve the specification of the aforementined subclauses, resolving at least some of the open issues.

2 Revision history

R0:

- Initial revision.

3 Implementation divergence

— GCC accepts a defining declaration without an *alignment-specifier* that happens to have the same alignment as specified in other declarations (in violation of [dcl.align]/6) (Compiler Explorer):

```
struct alignas(4) A;
struct A { alignas(4) int i; }; // GCC accepts, Clang, EDG, and MSVC
        reject
```

— Clang considers `alignas(0)` to have no effect for the purposes of [dcl.align]/5, yet considers it to have an effect for the purposes of [dcl.align]/6 (Compiler Explorer):

```
struct alignas(0) B1;
struct B1 {};               // Clang rejects, GCC, EDG, and MSVC accept

struct alignas(0) B2;
struct alignas(0) B2 {};    // all accept
```

— Clang rejects *alignment-specifier*s on non-static data members of reference type, EDG accepts only if the specified alignment is stricter than the alignment in the absence of *alignment-specifier*, GCC and MSVC accept any *alignment-specifier* (even though non-static data members that are references are not variables) (CWG3024) (Compiler Explorer):

```
struct C1 { alignas(2)  const int& r1 = 7; };    // Clang and EDG reject,
        GCC and MSVC accept
static_assert(alignof(C1) == 8);                 // all accept

struct C2 { alignas(16) const int& r2 = 7; };    // Clang rejects, GCC,
        EDG, and MSVC accept
static_assert(alignof(C2) == 16);                // all accept
```

— Clang and GCC prioritize `#pragma pack` over *alignment-specifier*s of non-static data members and their types, MSVC does the opposite, while EDG prioritize `#pragma pack` over *alignment-specifier* of types but not of non-static data members (Compiler Explorer):

```
struct alignas(8) D1 { int m; };

#pragma pack(2)
struct D2 {
  char m1;
  D1 m2;
  char m3;
  alignas(8) int m4;
};

static_assert(alignof(D2) == 2); // Clang and GCC accept
```

```
static_assert(alignof(D2) == 8); // EDG, MSVC, and Clang in MSVC compat-
        ibility mode accept

static_assert(sizeof(D2) == 12); // Clang and GCC accept
static_assert(sizeof(D2) == 24); // EDG accepts; m4 is at offset of 16
static_assert(sizeof(D2) == 32); // MSVC and Clang in MSVC compatibility
        mode accept
```

4 Issues with status quo wording

— CWG1211: presumably NAD, because [basic.life]/1 states that lifetime of an object cannot start until "storage with the proper alignment and size for type `T` is obtained". Maybe it's worth harmonizing "proper alignment", "sufficiently aligned", and "meets alignment requirements" across the Standard.

— CWG1617: [dcl.align]/6 doesn't handle non-defining declarations of a *type* that specify different alignment, when a definition of that type is not reachable (or they are not rechable from a definition).

— CWG2223: [dcl.align]/6 doesn't handle non-defining declarations of an *object* that specify different alignment, when a definition of that object is not reachable (or they are not rechable from a definition).

— CWG2840: presumably NAD, because [basic.align]/4 already specifies that alignment of `long double`, which is a fundamental type, is a valid alignment

— CWG3024: presumably NAD, because *alignment-specifiers* can be attached to variables and class declarations, while non-static data memebers that are references are not variables per [basic.pre]/7.

— [basic.align]/1 uses the verb "allocate" to describe alignment, even though this verb is quite tightly coupled with memory allocation and dynamic storage duration.

— [basic.align]/1 repeats [basic.life]/1 saying that creating an object in insufficiently aligned storage is undefined behavior.

— [basic.align]/2 specifies the semantics of `alignof` operator without leaving as much as a note in [expr.align].

— [basic.align]/3, [basic.align]/9, and [dcl.align]/2.2 repeat each other saying that support for extended alignments is implementation-defined, and that a program that needs an unsupported extended alignment is ill-formed.

— [basic.align]/4 and [basic.align]/5 use "valid alignment", but no other wording does.

— [basic.align]/6 doesn't require diagnostics when declarations are in different translation units but one is reachable from another.

— Definition of *alignment-specifier* is separated from [dcl.align].

5 Approach

This section is preliminary in the light of implementation divergence discussed above.

— Alignment is an integral value directly corresponding to layout of objects in memory.

— `alignof` expression reports alignment (which is what all implementations do in the presence of extensions).

— Alignment requirement is a property of types and objects specified by `alignas` and represented as an integral value.

— Two declarations of the same entity shall give it the same alignment.

To enable `#pragma pack` as an extension that a conforming implementation can have ([intro.compliance]/11):

— Correspondence between alignment and alignment requirement is implementation-defined (it seems that the status quo is that alignment requirement specifies the minimum alignment of an object).

— It is implementation-defined whether a given storage meets alignment requirement of a given object (so that the object's lifetime can start even if implementation places it in a storage that doesn't meen alignment requirement).

Hide removals: ☐
Strikethrough: ☑

6 Proposed wording

This section is preliminary in the light of implementation divergence discussed above.

§ 6.1 Alignment [basic.align]

1 ~~Object types have *alignment requirements* (6.9.2 [basic.fundamental], 6.9.4 [basic.compound]) which place restrictions on the addresses at which regions of storage an object of that type may be allocated.~~ An *alignment* is an ~~implementation-defined~~ integer value ~~representing the number of bytes between successive addresses at~~ which describes restrictions on the address of the first byte of a region of storage that a given object can ~~be allocated~~ occupy. Objects and object types have *alignment requirement* (6.9.2 [basic.fundamental], 6.9.4 [basic.compound]), which is a value of type `std::size_t`. An object type imposes an alignment requirement on every object of that type; stricter alignment can be requested using the *alignment-specifier* (9.13.2 [dcl.align]).

1a Alignment requirement of an entity influences its alignment in an implementation-defined manner. Consequently, it is implementation-defined whether a given region of storage meets alignment requirement of a given object.

[*Note:* ~~Attempting to create an object (6.8.2 [intro.object]) in storage that does not meet the alignment requirements of the object's type is undefined behavior.~~ The lifetime of an object cannot start if its storage is not sufficiently aligned (6.8.4 [basic.life]), including cases when the object is created by a *new-expression* whose allocation function is a non-allocating form (7.6.2.8 [expr.new], 17.6.3.4 [new.delete.placement]). — *end note*]

2 A *fundamental alignment requirement* is ~~represented by~~ an alignment requirement that is less than or equal to `alignof(std::max_align_t)`.

[*Note:* `alignof(std::max_align_t)` is the greatest alignment requirement supported by the implementation in all contexts~~, which is equal to `alignof(std::max_align_t)`~~ (17.2 [support.types]). — *end note*]

2a The alignment required for a type may be different when it is used as the type of a complete object and when it is used as the type of a subobject.

[*Example:*

```
struct B { long double d; };
struct D : virtual B { char c; };
```

When `D` is the type of a complete object, it will have a subobject of type `B`, so it must be aligned appropriately for a `long double`. If `D` appears as a subobject of another object that also has `B` as a virtual base class, the `B` subobject might be part of a different subobject, reducing the alignment requirements on the `D` subobject. — *end example*]

~~The result of the `alignof` operator reflects the alignment requirement of the type in the complete-object case.~~

3 An *extended alignment requirement* is ~~represented by~~ an alignment requirement greater than `alignof(std::max_align_t)`. It is implementation-defined whether any extended alignments are supported and the contexts in which they are supported (9.13.2 [dcl.align]). A type having an extended alignment requirement is an *over-aligned type*.

[*Note:* Every over-aligned type is ~~or contains a class type to which extended alignment applies (possibly through a non-static data member)~~ either specified to have an extended alignment requirement (9.13.2 [dcl.align]), or has a subobject S of type T, where at least one of S or T is specified to have an extended alignment requirement. — *end note*]

A *new-extended alignment* is ~~represented by~~ an alignment requirement greater than `__STDCPP_DEFAULT_NEW_ALIGNMENT__` (15.12 [cpp.predefined]).

4 ~~Alignments are represented as values of the type `std::size_t`. Valid alignments include only those values returned by an alignof expression for the fundamental types plus an additional implementation-defined set of values, which may be empty.~~ Every alignment ~~value~~ requirement shall be a non-negative integral power of two~~.~~, and shall be in one of the following sets of values:

(4.1) — values returned by an `alignof` expression whose *type-id* designates one of the fundamental types, or

(4.2) — an implementation-defined set of values.

5 Alignments have an order from *weaker* to *stronger* or *stricter* alignments. Stricter alignments have larger alignment values. An address that satisfies an alignment requirement also satisfies any weaker ~~valid~~ alignment requirement.

6 The alignment requirement of a complete type can be queried using an `alignof` expression (7.6.2.6 [expr.alignof]). Furthermore, the narrow character types (6.9.2 [basic.fundamental]) shall have the weakest alignment requirement.

[*Note:* The type `unsigned char` can be used as the element type of an array providing aligned storage (9.13.2 [dcl.align]). — *end note*]

7 Comparing alignments is meaningful and provides the obvious results:

(7.1) — Two alignments are equal when their numeric values are equal.

(7.2) — Two alignments are different when their numeric values are not equal.

(7.3) — When an alignment is larger than another it represents a stricter alignment.

8 [*Note:* The runtime pointer alignment function (20.2.5 [ptr.align]) can be used to obtain an aligned pointer within a buffer; an *alignment-specifier* (9.13.2 [dcl.align]) can be used to align storage explicitly. — *end note*]

9 ~~If a request for a specific extended alignment in a specific context is not supported by an implementation, the program is ill-formed.~~

§ 6.2 Alignof [expr.alignof]

1 An `alignof` expression yields the alignment that a hypothetical complete object of its operand type would have. The operand shall be a *type-id* representing a complete object type, or an array thereof, or a reference to one of those types.

[*Note:* Alignment can differ between a complete object and a subobject of the same type (6.8.3 [basic.align]). — *end note*]

2 The result is a prvalue of type `std::size_t`.

[*Note:* An `alignof` expression is an integral constant expression ([expr.const.const]). The *typedef-name* `std::size_t` is declared in the standard header `<cstddef>` (17.2.1 [cstddef.syn], 17.2.4 [support.types.layout]). — *end note*]

3 When `alignof` is applied to a reference type, the result is the alignment of the referenced type. When `alignof` is applied to an array type, the result is the alignment of the element type.

§ 6.3 Attribute syntax and semantics [dcl.attr.grammar]

Move the definition of *alignment-specifier* from paragraph 1 to the beginning of 9.13.2 [dcl.align]:

1 Attributes and annotations specify additional information for various source constructs such as types, variables, names, contract assertions, blocks, or translation units.

attribute-specifier-seq:
attribute-specifier attribute-specifier-seq$_{opt}$

attribute-specifier:
`[ [` *attribute-using-prefix*$_{opt}$ *attribute-list* `] ]`
`[ [` *annotation-list* `] ]`
alignment-specifier

~~*alignment-specifier*~~
~~`alignas (` *type-id* `...`$_{opt}$ `)`~~
~~`alignas (` *constant-expression* `...`$_{opt}$ `)`~~

[. . .]

§ 6.4 Alignment specifier [dcl.align]

Change the entire subclause 9.13.2 [dcl.align] as follows:

alignment-specifier
`alignas (` *type-id* `...`$_{opt}$ `)`
`alignas (` *constant-expression* `...`$_{opt}$ `)`

1 An *alignment-specifier* may be applied to ~~a variable or to a class data member, but it shall not be applied to a bit-field, a function parameter, or an *exception-declaration* (14.4 [except.handle]). An *alignment-specifier* may also be applied to the declaration of a class (in an *elaborated-type-specifier* (9.2.9.5 [dcl.type.elab]) or *class-head* (11 [class]), respectively).~~ a declaration of

(1.1) — a *class-name* (11.1 [class.pre], 9.2.9.5 [dcl.type.elab]) or

(1.2) — a variable, except for

(1.2.1) — variables introduced by declarations of function parameters,

(1.2.3) — bit-fields, and

(1.2.2) — *exception-declaration*s.

An *alignment-specifier* with an ellipsis is a pack expansion (13.7.4 [temp.variadic]).

2 When the *alignment-specifier* is of the form `alignas(` *constant-expression* `)`~~:~~ , the *constant-expression* shall be an integral constant expression that satisfies the requirements of an alignment requirement (6.8.3 [basic.align]).

(2.1) — ~~the *constant-expression* shall be an integral constant expression~~

(2.2) — ~~if the constant expression does not evaluate to an alignment value (6.8.3 [basic.align]), or evaluates to an extended alignment and the implementation does not support that alignment in the context of the declaration, the program is ill-formed.~~

3 An *alignment-specifier* of the form `alignas(` *type-id* `)` has the same effect as `alignas(alignof(` *type-id* `))`.

4 The alignment requirement of an entity is the strictest nonzero alignment specified by its *alignment-specifiers*, if any; otherwise, the *alignment-specifiers* have no effect.

5 The combined effect of all *alignment-specifiers* in a declaration shall not specify an alignment that is less strict than the alignment that would be required for the entity being declared if all *alignment-specifiers* appertaining to that entity were omitted.

[*Example:*

```
struct alignas(8) S {};
struct alignas(1) U {
  S s;
};  // error: U specifies an alignment that is less strict than if the alignas(1) were omitted.
```

— *end example*]

6 ~~If the defining declaration of an entity has an *alignment-specifier*, any non-defining declaration of that entity shall either specify equivalent alignment or have no *alignment-specifier*. Conversely, if any declaration of an entity has an *alignment-specifier*, every defining declaration of that entity shall specify an equivalent alignment. No diagnostic is required if declarations of an entity have different *alignment-specifier*s in different translation units.~~ If two declarations of an entity does not give it equivalent alignment requirement, the program is ill-formed; no diagnostic is required if neither declaration is reachable from the other.

[*Example:*

```
// Translation unit #1:
struct S { alignas(4) int x; } s, *p = &s;

// Translation unit #2:
struct alignas(~~16~~ 4) S;              // ill-formed, no diagnostic required: definition of S lacks ~~alignment~~ alignment-specifier
extern S* p;
```

— *end example*]

7 [*Example:* An aligned buffer with an alignment requirement of `A` and holding `N` elements of type `T` can be declared as:

```
alignas(T) alignas(A) T buffer[N];
```

Specifying `alignas(T)` ensures that the final requested alignment will not be weaker than `alignof(T)`, and therefore the program will not be ill-formed. — *end example*]

8 [*Example:*

```
alignas(double) void f();                                   // error:
          alignment applied to function
alignas(double) unsigned char c[sizeof(double)];      // array of
          characters, suitably aligned for a double
extern unsigned char c[sizeof(double)];               // no alig-
          nas necessary
alignas(float)
  extern unsigned char c[sizeof(double)];             // error: dif-
          ferent alignment in declaration
```

— *end example*]